Department of Computer Science and Engineering

B.Tech III Semester

Academic Year 2026-2027

Data Analysis Essentials(2501AI06)

Corner Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh - 533437

ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

DATA ANALYSIS ESSENTIALS

CERTIFICATE

This is to certify that the Cornerstone project work entitled "SMART RETAIL SALES ANALYSIS AND DEMAND FORECASTING" is submitted by

S L V KRISHNA REDDY	(25B11CS861)
P SURYA AKHIL	(25B11CS717)
K S R MOKSHAGNA	(25B11CS453)

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering the academic year 2026-2027.

Project Guide
M. Satyavathi, MCA
Assistant Professor,
Department of CSE

Head of the Department
Dr. A. Phani Sridhar, Ph.D.,
Assistant Professor & HOD
Department of CSE

DECLARATION

We hereby declare that the project entitled **"SMART RETAIL SALES ANALYSIS AND DEMAND FORECASTING"** is our original work and is submitted to **ADITYA UNIVERSITY, Surampalem**, in partial fulfillment of the requirements for the award of the **B.Tech. in Computer Science and Engineering.**

We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the **Institutional Academic Integrity and AI Usage Policy**.

Place: Surampalem

Date:

By
S L V KRISHNA REDDY (25B11CS861)
P SURYA AKHIL (25B11CS717)
K S R MOKSHAGNA (25B11CS453)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **M. Satyavathi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K Chandra Shekar, Associate Professor, Dr. Hari Jyothula, Associate Professor and Dr. P S Satya Prasad, Assistant Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. A Phani Sridhar, Assistant Professor & HOD**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

Retail businesses generate large volumes of sales data every day, making it essential to analyze this information for effective decision-making and business growth. The Smart Retail Sales Analysis and Demand Forecasting project is developed to analyze historical retail sales data, identify sales patterns, and perform predictive analysis using data analytics and machine learning techniques.

The project uses a DMart retail sales dataset covering the period from 2011 to 2017, containing information related to sales transactions, customers, products, categories, regions, quantities, discounts, and profits. Data preprocessing techniques such as data type conversion, handling missing values, feature engineering, categorical encoding, and feature scaling are applied to improve the quality and consistency of the dataset. Exploratory Data Analysis (EDA) is performed to understand sales distribution, yearly sales and profit trends, product and category performance, regional sales, customer segments, quantity patterns, and the relationship between discount and profit. Various visualizations, including bar graphs, pie charts, histograms, line charts, box plots, and scatter plots, are used to represent the analytical findings clearly.

For predictive analysis, Linear Regression, Decision Tree Regression, and Random Forest Regression models are trained using the processed dataset. The models are evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. The trained Random Forest model is saved using Python's pickle library and can be used to generate Sales predictions for new input data. The project therefore demonstrates the use of data preprocessing, exploratory analysis, visualization, and machine learning for understanding retail sales data and supporting data-driven business decisions.

Table of contents

S. No.	Contents	Pages
1	Title Page	i
2	Certificate	i
3	Declaration	ii
4	Acknowledgement	iii
5	Abstract	iv
6	Table of Contents	v
7	List of Figures	viii
8	List of Tables	ix
Chapter 1	**Introduction**	1 - 2
1.1	Introduction to the Project	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
Chapter 2	**Dataset Description and Understanding**	3 – 5
2.1	Dataset Source	3
2.2	Dataset Description	3
2.3	Features / Attributes	3
2.4	Target Variable, if applicable	5
2.5	Dataset Statistics / Initial Exploration	5
Chapter 3	**Data Preprocessing**	6 – 7
3.1	Data Collection / Loading	6
3.2	Handling Missing Values	6
3.3	Handling Duplicate / Inconsistent Data	6
3.4	Feature Scaling	6
3.5	Standardization	7
3.6	Label Encoding	7
3.7	One-Hot Encoding	7

3.8	Other Project-Specific Preprocessing	7
Chapter 4	**System Design and Methodology**	8 – 11
4.1	Proposed Methodology	8
4.2	System Workflow / Architecture	9
4.3	Flowchart	10
4.4	Tools and Technologies Used	10
4.5	Algorithm / Model Used	11
Chapter 5	**Implementation**	12 – 14
5.1	Development Environment	12
5.2	Dataset Loading and Exploration	12
5.3	Preprocessing Implementation	12
5.4	Exploratory Data Analysis (EDA)	12
5.5	Model / Analysis Implementation	13
5.6	Important Code Explanation	13
Chapter 6	**Data Visualization and Results**	15 – 18
6.1	Bar Graph	15
6.2	Pie Chart	16
6.3	Histogram	16
6.4	Box Plot	17
6.5	Line Chart	17
6.6	Scatter Plot	18
6.7	Project-Specific Visualizations	18
6.8	Interpretation of Visualizations	18
Chapter 7	**Model Evaluation / Analysis of Results**	19 – 20
7.1	Evaluation Metrics	19
7.2	Experimental Results	19
7.3	Result Interpretation	19
7.4	Discussion	20
Chapter 8	**Output Screenshots**	21 – 25
8.1	Input / Dataset Screenshots	21

8.2 Preprocessing Outputs 22
8.3 Visualization Outputs 23
8.4 Prediction / Analysis Outputs 25
Chapter 9 Conclusion and Future Enhancements 26 – 27
9.1 Conclusion 26
9.2 Limitations 26
9.3 Future Enhancements 27
References 28
Appendix – Source Code 29 - 40

List of Figures

S.NO	FIGURE NO	NAME OF THE FIGURE	PAGE NO
1	4.2.1	Overall Project Workflow	9
2	4.2.2	Predictive Modelling and Web Prediction Flow	9
3	4.3.1	Project Flowchart	10
4	6.1.1	Total Sales by Year	15
5	6.1.2	Total Profit by Order Year	15
6	6.2.1	Product Category Distribution	16
7	6.3.1	Distribution of Sales	16
8	6.4.1	Boxplot for Sales	17
9	6.5.1	Line Chart for Quantity and Average Sales	17
10	6.6.1	Scatter Plot of Discount VS Average Profit	18
11	6.7.1	Project-Specific Visualizations	18
12	8.1.1	Dataset	21
13	8.1.2	First Records of the Dataset	21
14	8.1.3	Dataset Shape, columns & Information Output	22
15	8.2.1	Preprocessing Discount Column	22
16	8.2.2	Adding a New Column	23
17	8.3.1	Sales by Category	23
18	8.3.2	Sales by Region	24
19	8.3.3	Interface of Website	24
20	8.4.1	Interface of Predicting Model	25

LIST OF TABLES

S.NO	TABLE NO	NAME OF THE TABLE	PAGE NO
1	2.3.1	Description about Dataset Attributes	3
2	4.4.1	Tools and Technologies Used in the Proposed System	10
3	7.1.1	Evaluation Metrics and Their Purpose	19
4	7.2.1	Models Used in the Proposed System	19

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

Retail businesses generate large volumes of transactional data through daily sales activities. This information contains useful evidence about product demand, customer segments, regional performance, sales trends, profitability, discounts, and purchasing patterns. However, raw transactional data is not directly suitable for decision-making because it requires systematic cleaning, transformation, analysis, and visualization.

The Smart Retail Sales Analysis and Demand Forecasting project applies a complete data analytics workflow to retail sales records. The implementation documented in the project notebook uses a DMart sales dataset covering the period from 2011 to 2017. The analysis begins with data loading and exploration, followed by data cleaning, feature engineering, exploratory data analysis, preprocessing, model training, model evaluation, and prediction.

The project combines descriptive analysis with predictive modeling. Sales, profit, product, category, segment, region, quantity, and discount information are examined through statistical summaries and visualizations. Machine-learning models are then trained to estimate Sales from the prepared features. The Random Forest model is serialized for reuse in the prediction stage.

The project also connects the analytical workflow with dashboarding and a web-based prediction interface as described in the project abstract and review material. The documentation distinguishes these project deliverables from the notebook outputs so that unsupported numerical results or deployment claims are not introduced.

1.2 Problem Statement

Retail organizations need timely and reliable information about sales and demand to support inventory planning, product management, and business decisions. Manual examination of large transaction datasets can be time-consuming and may make it difficult to identify trends, relationships, and product-level patterns. Historical sales records also require appropriate transformation before they can be used for predictive modeling.

The project addresses the problem of converting raw retail transaction data into a cleaned analytical dataset, extracting meaningful sales insights, training predictive models for Sales estimation, and presenting the results in a form that can support data-driven retail decision-making.

The major challenges considered in the project include:

- Large volumes of historical transaction information.
- Missing or incomplete values in selected fields.
- Different data types and categorical variables requiring preprocessing.
- Difficulty in identifying sales and profit patterns from raw records.

- Need for a consistent machine-learning feature structure.
- Need to communicate analytical and predictive results in an understandable form.

1.3 Objectives of the Project

- To collect and load the retail sales dataset into a Python-based analysis environment.
- To explore dataset structure, data types, missing values, and descriptive statistics.
- To clean missing, inconsistent, and improperly formatted data.
- To create useful date-based and business-oriented features through feature engineering.
- To perform exploratory data analysis of sales, profit, products, categories, customer segments, and regions.
- To prepare the processed dataset for machine learning.
- To train Linear Regression, Decision Tree Regressor, and Random Forest Regressor models.
- To evaluate the trained models using MAE, RMSE, and R^2.
- To save the Random Forest model for reuse in prediction.
- To provide analytical and predictive outputs that can support data-driven retail decisions.

1.4 Scope of the Project

The scope of the project includes retail sales data collection, data exploration, data cleaning, feature engineering, exploratory data analysis, visualization, machine-learning preparation, regression model training, model evaluation, model serialization, and prediction using new input data.

- Historical sales analysis for the DMart dataset covering 2011–2017.
- Product, category, sub-category, customer-segment, and regional analysis.
- Sales, profit, quantity, discount, and date-based analysis.
- Feature creation such as Order Year, Order Month, Order Day, Order Quarter, Shipping Days, Discount Amount, Profit Margin, Has Discount, and Sales Category.
- Categorical encoding and numerical scaling for model preparation.
- Regression-based Sales prediction.
- Dashboard and web-prediction integration as described in the project deliverables.

The project does not represent a dedicated chronological multi-step time-series forecasting implementation. The current notebook uses Sales as the prediction target and a random train-test split. Dedicated time-series forecasting and chronological validation are therefore treated as future enhancements.

CHAPTER 2

DATASET DESCRIPTION AND UNDERSTANDING

2.1 Dataset Source

The project uses a DMart retail sales dataset covering the period from 2011 to 2017. The project notebook identifies the dataset as the "DMart Sales Dataset (2011–2017)" and loads the Excel workbook DMart_All_Data_2011_2017.xlsx from the All Data sheet. The dataset contains transaction-level information required for sales, profit, product, customer, regional, and discount analysis.

2.2 Dataset Description

The dataset contains retail transaction records with information related to orders, customers, products, dates, quantities, sales, discounts, profits, categories, sub-categories, regions, and other attributes. The data is first loaded into a Pandas DataFrame and then examined for structure, data types, missing values, and statistical properties.

The major data groups used in the analysis are:

- Order and transaction information: Row ID, Order ID, Order Date, Ship Date.
- Customer information: Customer ID, Customer Name, Segment.
- Geographical information: Country, City, State, Region.
- Product information: Product ID, Product Name, Category, Sub-Category.
- Sales measures: Sales, Quantity, Discount, Profit.

2.3 Features / Attributes

Attribute	Description
Row ID	Unique row identifier in the source transaction data
Order ID	Identifier of the customer order
Order Date	Date on which the order was placed
Ship Date	Date on which the order was shipped
Customer ID	Customer identifier
Customer Name	Customer name
Segment	Customer segment
Country	Country associated with the transaction
City	City associated with the transaction

State	State associated with the transaction
Region	Regional classification
Product ID	Product identifier
Product Name	Name of the product
Category	Product category
Sub-Category	Product sub-category
Sales	Sales value and prediction target
Quantity	Number of units sold
Discount	Discount value/rate in the transaction
Profit	Profit value
Source	Source field where available
Order_Year	Year extracted from Order Date
Order_Month	Month extracted from Order Date
Order_Day	Day extracted from Order Date
Order_Quarter	Quarter extracted from Order Date
Shipping_Days	Number of days between Order Date and Ship Date
Discount_Amount	Sales multiplied by Discount
Profit_Margin	Profit relative to Sales, expressed as a percentage
Has_Discount	Boolean indicator for whether Discount is greater than zero
Sales_Category	Low, Medium, or High sales category derived from Sales

Table 2.3.1: Description about Dataset Attributes

2.4 Target Variable, if applicable

For the predictive implementation in the project notebook, the target variable is Sales. The machine-learning dataset is created by separating Sales from the remaining processed features. The target therefore represents the transaction Sales value to be estimated by the regression models.

The project title refers to demand forecasting; however, the supplied notebook implementation estimates Sales rather than implementing a dedicated future-period time-series demand target. This distinction is important when interpreting the current model and is considered in the limitations and future-enhancement sections.

2.5 Dataset Statistics / Initial Exploration

Initial exploration is performed before detailed analysis. The notebook uses Pandas operations to display the first and last records, inspect dataset dimensions, examine column names and data types, generate descriptive statistics, inspect categorical values, and identify missing values.

```
df.head()
df.tail()
df.shape
df.columns
df.dtypes
df.info()
df.describe()
df.describe(include="object")
df.isnull().sum()
```

The actual numerical statistics should be taken directly from the final executed notebook output. The supplied notebook contains the exploration code but this documentation does not invent row counts, summary values, or other numerical statistics that are not available in the source material.

CHAPTER 3

DATA PREPROCESSING

3.1 Data Collection / Loading

The dataset is loaded into the Python environment using the Pandas library. The project notebook reads the All Data worksheet from the DMart Excel workbook.

```
import pandas as pd
df = pd.read_excel("DMart_All_Data_2011_2017.xlsx", sheet_name="All Data")
print(df.head())
```

After loading, the dataset is inspected to understand its structure before detailed preprocessing and analysis.

3.2 Handling Missing Values

Missing values are checked for important attributes before analysis. The preprocessing implementation uses different strategies according to the field type. Segment and Region missing values are filled using the mode. Sales and Profit missing values are filled using the median. Quantity is converted to numeric form, missing values are filled with the median, and the result is converted to integer type. Discount is converted to numeric form and missing values are filled using the median.

Order Date is converted to datetime using error coercion. Ship Date is also converted to datetime when the field is available. These operations help create reliable date-based features and shipping-duration information.

3.3 Handling Duplicate / Inconsistent Data

The project checks the structure and values of categorical fields such as Category and Sub-Category and inspects important variables for consistency. Identifier columns are removed from the model feature set because they do not represent useful predictive information for the target Sales. The notebook does not document a separate printed duplicate-count result; therefore, no numerical duplicate count is asserted here.

3.4 Feature Scaling

Feature scaling is applied to numerical variables used by the predictive model. The notebook uses StandardScaler for Quantity, Discount, Order_Month, Order_Day, Order_Quarter, and Shipping_Days. Scaling places these variables on a comparable numerical scale for model processing.

3.5 Standardization

Standardization transforms a numerical variable using its mean and standard deviation. The formula is:

$Z = (X - \mu) / \sigma$

where X is the original value, μ is the mean of the variable, and σ is its standard deviation. In the project implementation, StandardScaler performs this transformation on the selected numerical feature columns.

3.6 Label Encoding

Categorical attributes are converted into numerical labels using LabelEncoder. The notebook applies label encoding to available categorical columns including Ship Mode, Segment, Country, City, State, Region, Category, Sub-Category, and Sales_Category. This allows categorical information to be represented numerically for machine-learning processing.

3.7 One-Hot Encoding

One-hot encoding is a common categorical preprocessing method in which each category becomes a separate binary feature. The supplied notebook does not use pd.get_dummies for the final preprocessing pipeline; instead, it uses LabelEncoder for the available categorical columns. Therefore, one-hot encoding is described here as a standard preprocessing concept and not as a claimed final operation of the supplied notebook.

3.8 Other Project-Specific Preprocessing

- Identifier columns Row ID, Order ID, Customer ID, and Product ID are removed from the model features.
- Date fields are transformed into Order Year, Order Month, Order Day, and Order Quarter.
- Shipping Days is calculated as the difference between Ship Date and Order Date.
- Discount Amount is calculated as Sales multiplied by Discount.
- Profit Margin is calculated from Profit and Sales with zero-handling for zero Sales values.
- Has Discount is created as a Boolean indicator based on whether Discount is greater than zero.
- Sales Category is derived using Low for Sales up to 100, Medium for Sales up to 500, and High for values above 500.
- Unused text and target-related fields are removed from the model input according to the notebook preprocessing stage.
- Infinite values are replaced with missing values and remaining missing values are filled with zero before model training.

CHAPTER 4

SYSTEM DESIGN AND METHODOLOGY

4.1 Proposed Methodology

The project follows a systematic data analysis and predictive modeling methodology. The major stages are:

- Data Collection: obtain the DMart retail sales dataset.
- Data Loading: load the Excel data into a Pandas DataFrame.
- Data Exploration: inspect shape, columns, data types, missing values, and statistical summaries.
- Data Cleaning: handle missing values and convert fields to appropriate types.
- Feature Engineering: derive date, shipping, discount, margin, discount-indicator, and sales-category features.
- Exploratory Data Analysis: analyze sales, profit, categories, products, segments, regions, discounts, and quantities.
- Data Preprocessing: encode categorical values, remove unsuitable fields, and scale selected numerical variables.
- Model Training: train Linear Regression, Decision Tree Regressor, and Random Forest Regressor.
- Model Evaluation: calculate MAE, RMSE, and R^2 for the models.
- Model Saving: serialize the Random Forest model using pickle.
- Prediction: prepare new input data in the same feature structure and generate a Sales prediction.

4.2 System Workflow / Architecture

The project architecture represents the flow from the raw retail dataset to analytical outputs and predictive results.

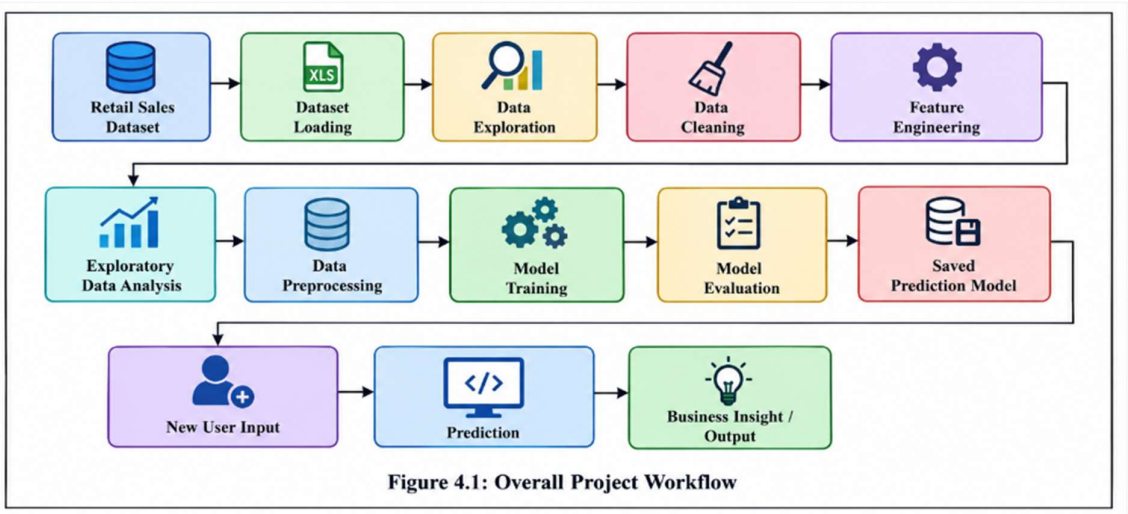


Figure 4.2.1: Overall Project Workflow

The planned web prediction flow described in the review material is:

User Input → HTML/CSS/JavaScript Frontend → Flask Backend → Trained Python Model → Prediction Result

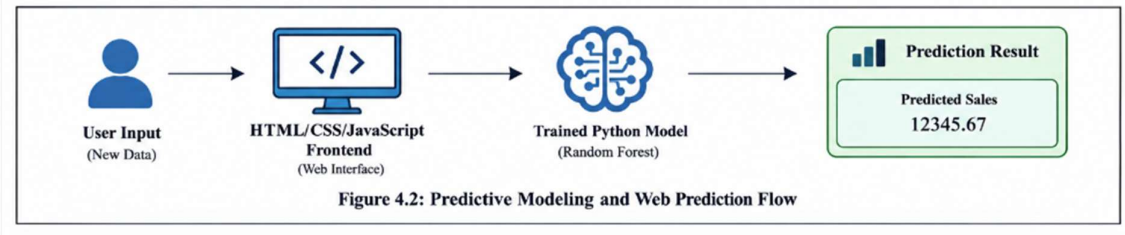


Figure 4.2.2: Predictive Modeling and Web Prediction Flow

4.3 Flowchart

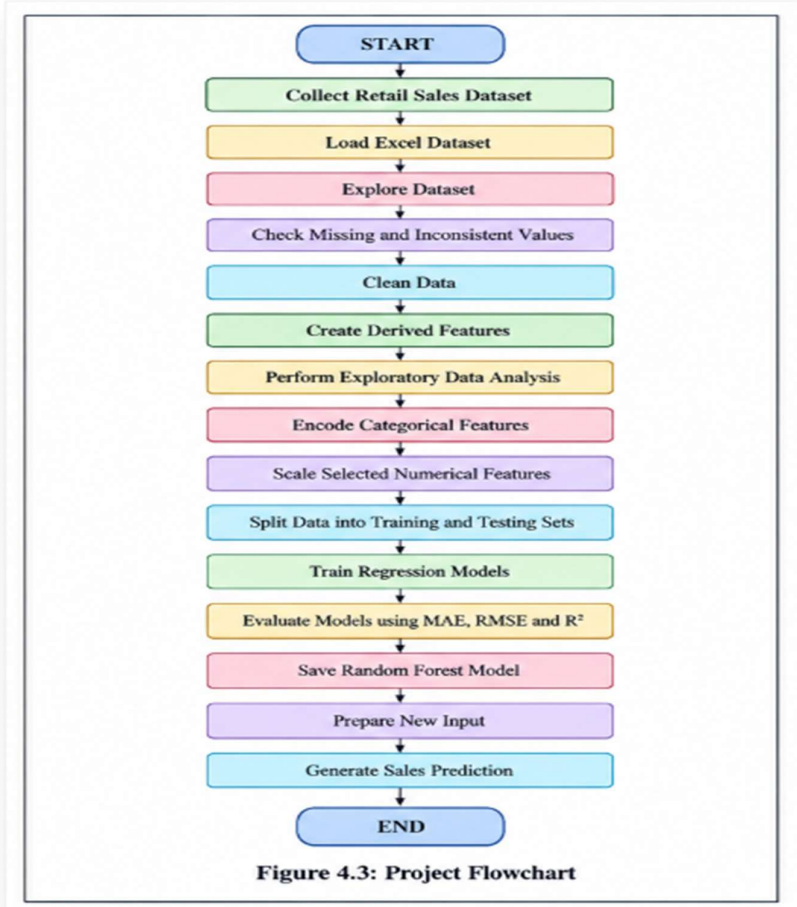


Figure 4.3.1: Project Flowchart

4.4 Tools and Technologies Used

Tool / Technology	Purpose
Python	Programming language for data analysis and machine learning
Jupyter Notebook	Development and experimentation environment used for the project notebook
Pandas	Data loading, cleaning, transformation, grouping, and analysis
NumPy	Numerical operations and array processing
Matplotlib	Data visualization and graphical analysis

Seaborn	Statistical visualization where used
Scikit-learn	Preprocessing, train-test split, regression models, and evaluation metrics
Excel	Source dataset storage in the DMart workbook
Flask	Backend technology specified for the prediction website
HTML / CSS / JavaScript	Frontend technologies specified for the prediction interface
Pickle	Serialization of the trained Random Forest model

Table 4.4.1: Tools and Technologies Used in the Proposed System

4.5 Algorithm / Model Used

The predictive implementation uses supervised regression because the target variable Sales is numerical. Three models are trained using the same processed feature set and train-test split.

- Linear Regression: a baseline regression model that estimates Sales as a linear combination of the input features.
- Decision Tree Regressor: a nonlinear regression model that creates decision rules through tree-based splits.
- Random Forest Regressor: an ensemble regression model that combines predictions from multiple decision trees. The notebook saves this model as dmart_sales_model.pkl for reuse in prediction.

The notebook uses train_test_split with test_size=0.2 and random_state=42. The evaluation metrics are Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score.

CHAPTER 5

IMPLEMENTATION

5.1 Development Environment

The implementation is developed in Python using a Jupyter Notebook workflow. The project uses Pandas and NumPy for data processing, Matplotlib and Seaborn for visualization, and Scikit-learn for preprocessing, model training, and evaluation. The project review also specifies Power BI for dashboarding and Flask with HTML, CSS, and JavaScript for the prediction web application.

5.2 Dataset Loading and Exploration

```
import pandas as pd
df = pd.read_excel("DMart_All_Data_2011_2017.xlsx", sheet_name="All Data")
df.head()
df.tail()
df.shape
df.columns
df.dtypes
df.info()
df.describe()
df.isnull().sum()
```

These operations establish the initial structure of the dataset and identify fields that require cleaning or transformation.

5.3 Preprocessing Implementation

The preprocessing implementation converts date fields, fills missing values, creates engineered features, encodes categorical variables, removes unsuitable columns, replaces infinite values, fills remaining missing values, and scales selected numerical features. The processing sequence is designed so that the same feature structure can be recreated when a new input is supplied to the saved model.

5.4 Exploratory Data Analysis (EDA)

- Overall sales statistics including total, average, minimum, and maximum Sales.
- Total Profit and quantity-related summaries.
- Sales distribution using a histogram.
- Sales and Profit trends by Order Year.
- Product category distribution.
- Sub-category distribution and top products by Sales.
- Sales and Profit by Category.
- Sales and Profit by Region.
- Segment distribution and average Sales/Profit by Segment.

- Discount versus Profit analysis.
- Category versus Region sales cross-tabulation.
- Segment versus Category sales cross-tabulation.
- Average Sales by Quantity.

5.5 Model / Analysis Implementation

The machine-learning implementation separates Sales as the target variable and uses the processed remaining columns as model inputs. The data is split into training and testing subsets using a fixed random state. Three regression models are trained and evaluated using the same evaluation metrics.

```
X = df.drop("Sales", axis=1)
y = df["Sales"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model_lr = LinearRegression()
model_dt = DecisionTreeRegressor(random_state=42)
model_rf = RandomForestRegressor(n_estimators=100, random_state=42)
```

5.6 Important Code Explanation

Reading Dataset

```
pd.read_excel("DMart_All_Data_2011_2017.xlsx", sheet_name="All Data")
```

This statement loads the All Data worksheet of the DMart Excel workbook into a Pandas DataFrame.

Checking Missing Values

```
df.isnull().sum()
```

This operation displays the number of missing values in each column.

Creating Date Features

```
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
```

This converts Order Date into a proper datetime representation and coerces invalid values to missing.

Creating Shipping Days

```
df["Shipping_Days"] = (df["Ship Date"] - df["Order Date"]).dt.days
```

This calculates the number of days between ordering and shipping.

Training the Random Forest

model_rf.fit(X_train, y_train)

This trains the Random Forest regression model using the training data.

Evaluating the Model

mean_absolute_error(y_test, y_pred)

This calculates the average absolute difference between actual and predicted Sales values.

Saving the Model

pickle.dump(model_rf, file)

This serializes the trained Random Forest model so that it can be reused during prediction.

CHAPTER 6

DATA VISUALIZATION AND RESULTS

6.1 Bar Graph

Bar graphs are used to compare categorical groups such as product categories, regions, sub-categories, and top products. In this project, category-wise and region-wise Sales and Profit analysis provides a direct comparison of retail performance across groups.

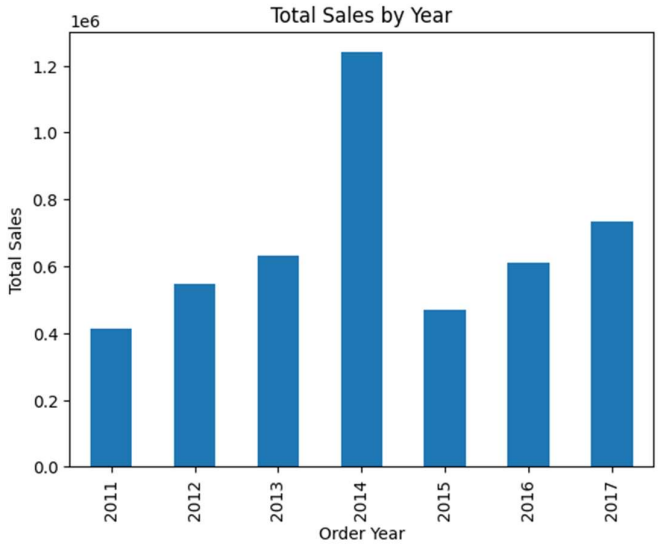


Figure 6.1.1: Total Sales by Year

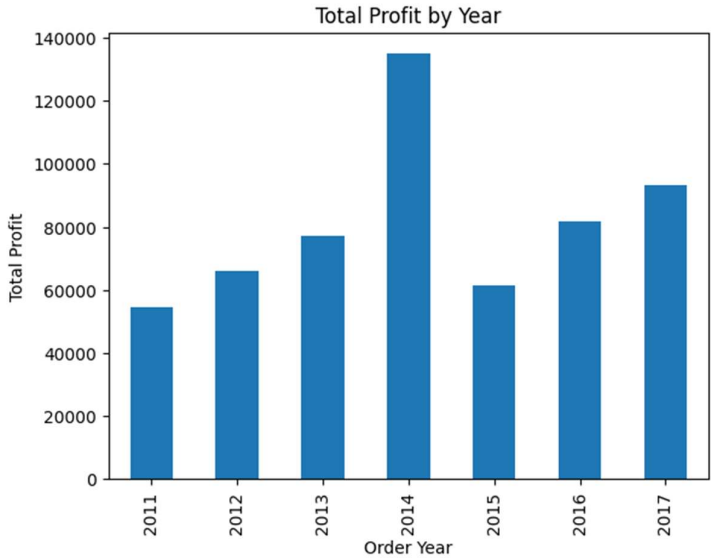


Figure 6.1.2: Total Profit by Order Year

6.2 Pie Chart

Pie charts are useful for showing proportions. The project uses category distribution and segment distribution visualizations to communicate how transactions are distributed across the available groups.

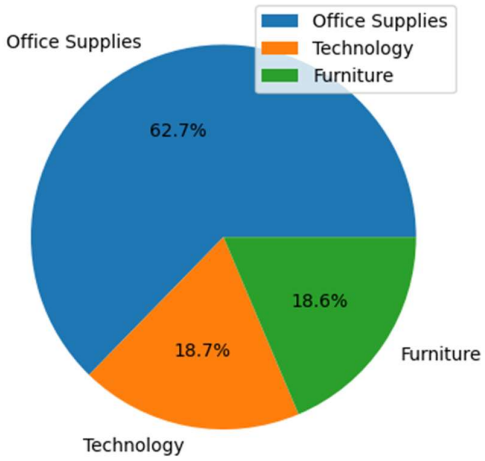


Figure 6.2.1: Product Category Distribution

6.3 Histogram

A histogram is used to understand the distribution of a numerical variable. The project uses a Sales distribution histogram to examine how transaction Sales values are distributed.

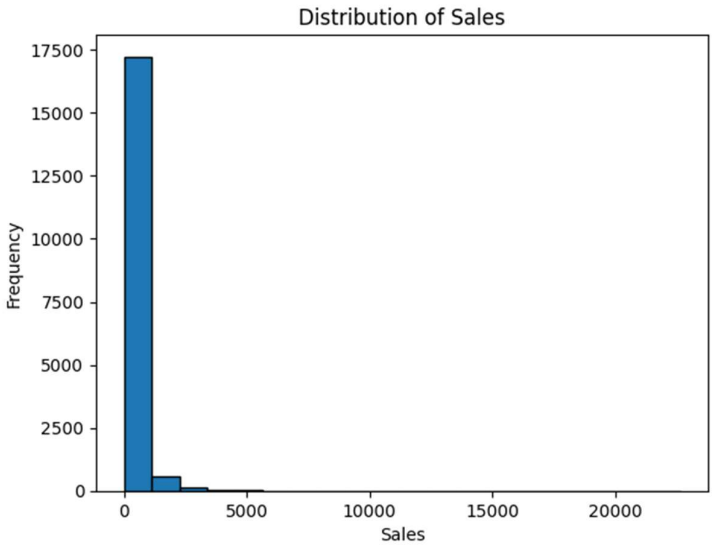


Figure 6.3.1: Distribution of Sales

6.4 Box Plot

Box plots are useful for examining the spread of numerical data and identifying possible outliers. The notebook uses a Sales box plot during data exploration to inspect the distribution and extreme values of Sales.

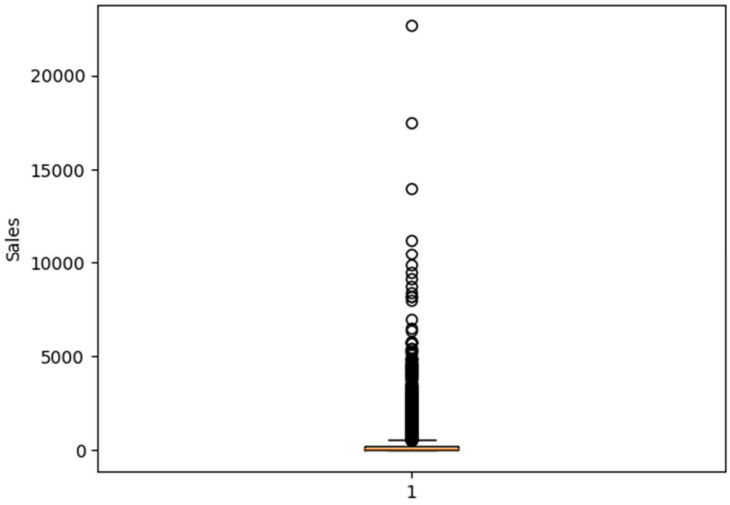


Figure 6.4.1: Boxplot for Sales

6.5 Line Chart

Line charts are appropriate for showing changes over ordered values. The project uses time-based analysis such as Sales and Profit by Order Year and average Sales by Quantity to communicate trends or ordered relationships.

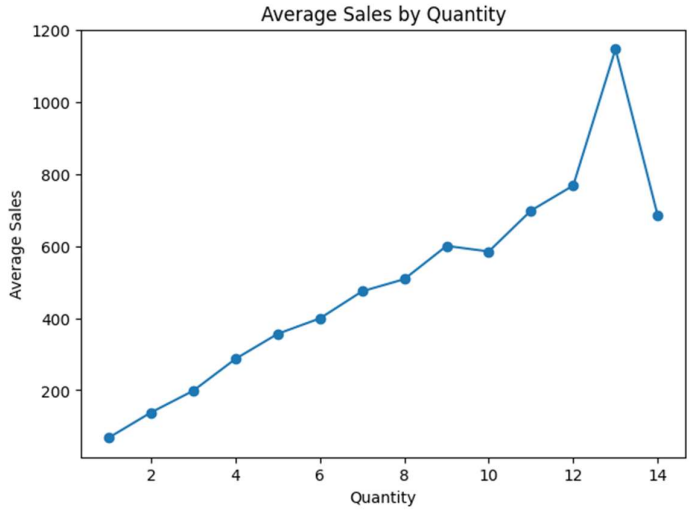


Figure 6.5.1: Line Chart for Quantity and Average Sales

6.6 Scatter Plot

Scatter plots are used to examine relationships between numerical variables. The project analyzes relationships such as Discount versus Profit and other numerical relationships generated during EDA.

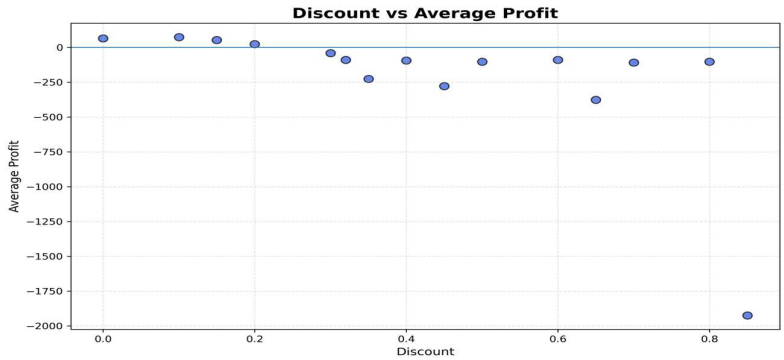


Figure 6.6.1: Scatter Plot of "Discount VS Average Profit"

6.7 Project-Specific Visualizations

In addition to the standard EDA plots, the project includes specific visualizations that provide business-focused insights. These visualizations help in understanding sales performance, customer behaviour, and product trends, which are useful for data-driven decision making.



Figure 6.7.1: Project-Specific Visualizations

6.8 Interpretation of Visualizations

The visualizations transform numerical results into graphical patterns that are easier to interpret. The Sales distribution histogram provides information about the spread of transaction values. Year-wise Sales and Profit charts provide a view of changes across the available years. Category and regional charts support comparison of retail performance across groups. The remaining EDA visualizations provide additional information about products, segments, discounts, quantities, and relationships between variables.

Specific numerical interpretations such as the highest-performing category, exact annual Sales value, or exact regional contribution should be taken from the final executed notebook output or dashboard rather than assumed from the methodology alone.

CHAPTER 7

MODEL EVALUATION / ANALYSIS OF RESULTS

7.1 Evaluation Metrics

Metric	Purpose
Mean Absolute Error (MAE)	Measures the average absolute difference between actual and predicted Sales.
Root Mean Squared Error (RMSE)	Measures prediction error while giving greater weight to larger errors.
R^2 Score	Indicates the proportion of variation in the target that is explained by the regression model.

Table 7.1.1: Evaluation Metrics and Their Purpose

7.2 Experimental Results

Model	Role in the Project
Linear Regression	Baseline regression model for estimating Sales.
Decision Tree Regressor	Nonlinear regression model based on decision-tree splits.
Random Forest Regressor	Ensemble regression model using multiple decision trees; saved for prediction.

Table 7.2.1: Models Used in the Proposed System

The supplied notebook contains the implementation required to calculate MAE, RMSE, and R^2 for all three models. However, the supplied notebook file does not contain the printed numerical metric outputs. Therefore, exact performance values are not invented in this documentation. The final numerical values should be copied from the executed notebook output when available.

7.3 Result Interpretation

The evaluation metrics provide a common basis for examining the predictive behavior of the three regression models. Lower MAE and RMSE indicate smaller prediction errors, while an R^2 value closer to 1 indicates that a larger proportion of target variation is explained by the model. The actual model comparison must be made from the numerical outputs generated by the project execution.

The Random Forest model is serialized after training and is used in the notebook prediction stage. This identifies it as the model selected for reuse in the supplied implementation, but the documentation does not claim that it is numerically superior without the actual evaluation results.

7.4 Discussion

The modeling workflow provides a direct comparison between a simple linear baseline, a single decision-tree model, and an ensemble Random Forest model. The same train-test split and evaluation measures are used to maintain consistency across the experiments. The resulting model can be reused for new input data after the input is transformed into the same feature structure used during training.

An important limitation is that the current implementation predicts transaction-level Sales using a random train-test split. A dedicated demand forecasting system for future periods would require chronological splitting and time-series-oriented validation. This is considered in the future-enhancement plan.

CHAPTER 8

OUTPUT SCREENSHOTS

8.1 Input / Dataset Screenshots

The following screenshots should be included from the actual executed project notebook:

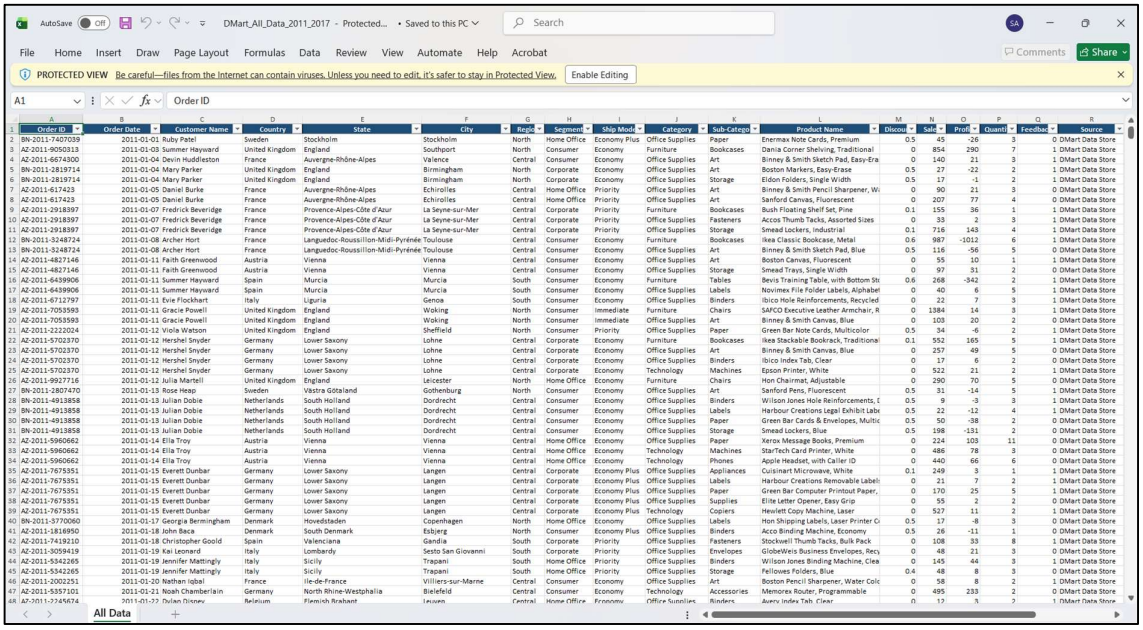


Figure 8.1.1: Dataset

```
df.head()
```

	Order ID	Order Date	Customer Name	Country	State	City	Region	Segment	Ship Mode	Category	Sub-Category	Product Name	Discount	Sales	Profit	Quantity	Fee
0	BN-2011-7407039	2011-01-01	Ruby Patel	Sweden	Stockholm	Stockholm	North	Home Office	Economy Plus	Office Supplies	Paper	Enermax Note Cards, Premium	0.5	45.0	-26.0	3	
1	AZ-2011-9050313	2011-01-03	Summer Hayward	United Kingdom	England	Southport	North	Consumer	Economy	Furniture	Bookcases	Dania Corner Shelving, Traditional	0.0	854.0	290.0	7	
2	AZ-2011-6674300	2011-01-04	Devin Huddleston	France	Auvergne-Rhône-Alpes	Valence	Central	Consumer	Economy	Office Supplies	Art	Binney & Smith Sketch Pad, Easy-Erase	0.0	140.0	21.0	3	
3	BN-2011-2819714	2011-01-04	Mary Parker	United Kingdom	England	Birmingham	North	Corporate	Economy	Office Supplies	Art	Boston Markers, Easy-Erase	0.5	27.0	-22.0	2	
4	BN-2011-2819714	2011-01-04	Mary Parker	United Kingdom	England	Birmingham	North	Corporate	Economy	Office Supplies	Storage	Eldon Folders, Single Width	0.5	17.0	-1.0	2	

Figure 8.1.2: First records of the dataset

```
[4]: df.shape

[4]: (18041, 23)

[5]: df.columns

[5]: Index(['Order ID', 'Order Date', 'Customer Name', 'Country', 'State', 'City',
       'Region', 'Segment', 'Ship Mode', 'Category', 'Sub-Category',
       'Product Name', 'Discount', 'Sales', 'Profit', 'Quantity', 'Feedback?',
       'Source', 'Row ID', 'Ship Date', 'Customer ID', 'Postal Code',
       'Product ID'],
      dtype='object')

[6]: df.dtypes

[6]: Order ID                 object
     Order Date       datetime64[ns]
     Customer Name             object
     Country                   object
     State                     object
     City                      object
     Region                    object
     Segment                   object
     Ship Mode                 object
     Category                  object
     Sub-Category              object
     Product Name              object
     Discount                 float64
     Sales                    float64
     Profit                   float64
     Quantity                   int64
     Feedback?                float64
     Source                    object
     Row ID                   float64
     Ship Date        datetime64[ns]
     Customer ID               object
     Postal Code              float64
     Product ID                object
     dtype: object
```

Figure 8.1.3: Dataset shape, columns and information output

8.2 Preprocessing Outputs

Handling Discount

```
: df["Discount"].unique()[:20]

: array([0.5 , 0.  , 0.1 , 0.6 , 0.4 , 0.2 , 0.65, 0.15, 0.7 , 0.3 , 0.35,
       0.8 , 0.85, 0.45, 0.32])

: df["Discount"] = pd.to_numeric(df["Discount"], errors="coerce")
  df["Discount"].isnull().sum()

: np.int64(0)

: df["Discount"] = df["Discount"].fillna(df["Discount"].median())
  df["Discount"].unique()[:20]

: array([0.5 , 0.  , 0.1 , 0.6 , 0.4 , 0.2 , 0.65, 0.15, 0.7 , 0.3 , 0.35,
       0.8 , 0.85, 0.45, 0.32])
```

Figure 8.2.1: Preprocessing Discount column.

Adding Discount Amount

```
df["Discount_Amount"] = df["Sales"] * df["Discount"]
df["Discount_Amount"].head()
```

```
0    22.5
1     0.0
2     0.0
3    13.5
4     8.5
Name: Discount_Amount, dtype: float64
```

Adding Profit Margin

```
df["Profit_Margin"] = (df["Profit"] / df["Sales"].replace(0, pd.NA)) * 100
df["Profit_Margin"] = df["Profit_Margin"].fillna(0)
df["Profit_Margin"].head()
```

```
0   -57.777778
1    33.957845
2    15.000000
3   -81.481481
4    -5.882353
Name: Profit_Margin, dtype: float64
```

Figure 8.2.2: Adding a new column.

8.3 Visualization Outputs

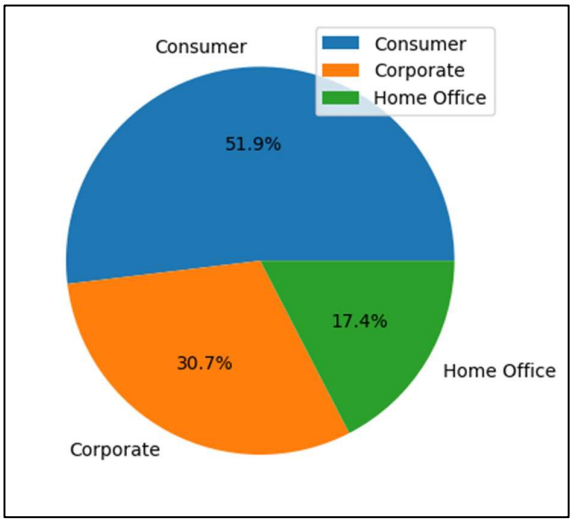


Figure 8.3.1: Sales by Category

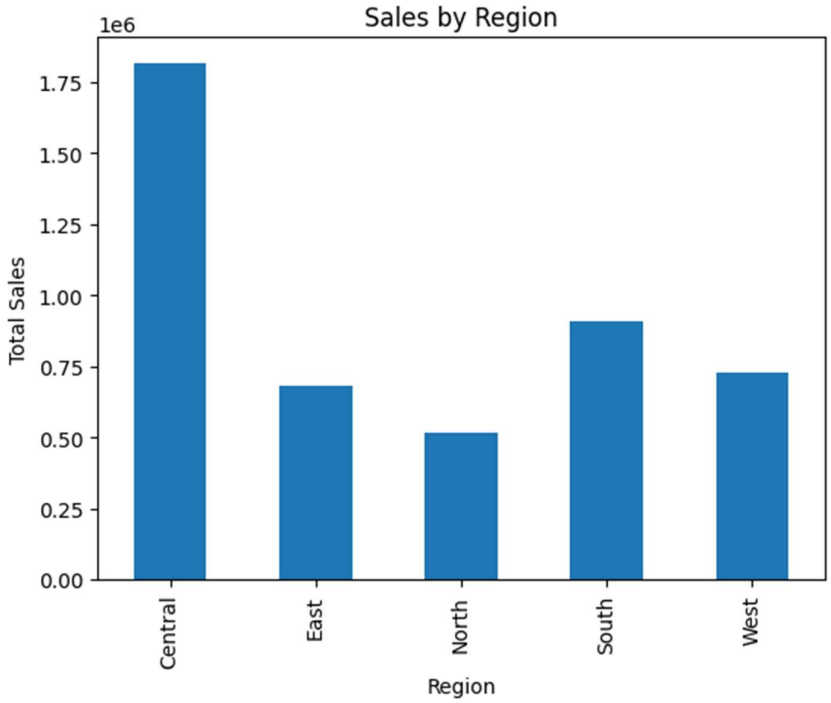


Figure 8.3.2: Sales by Region

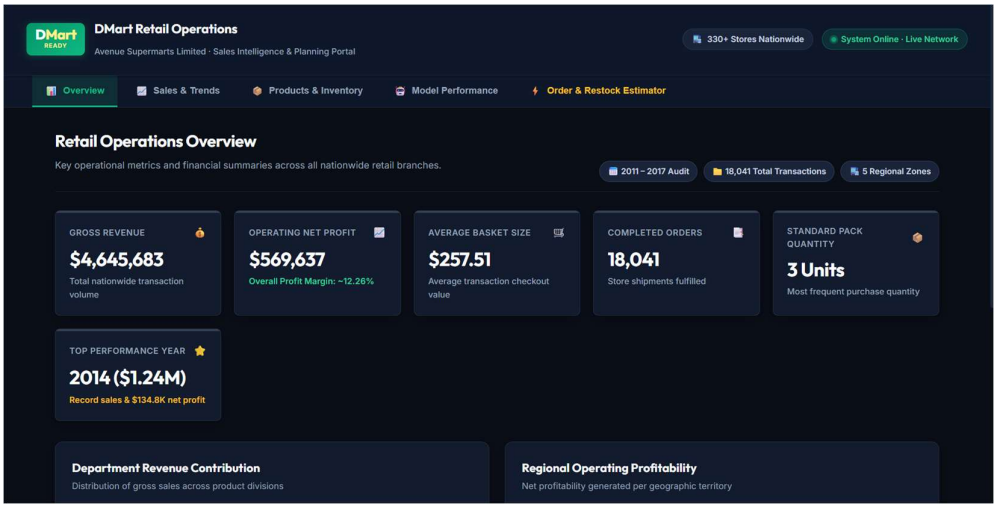


Figure 8.3.3: Interface of website

8.4 Prediction / Analysis Outputs

The prediction stage of the notebook creates a new input record, aligns it to the model feature structure, applies the same scaling procedure, and generates a predicted Sales value using the saved Random Forest model.

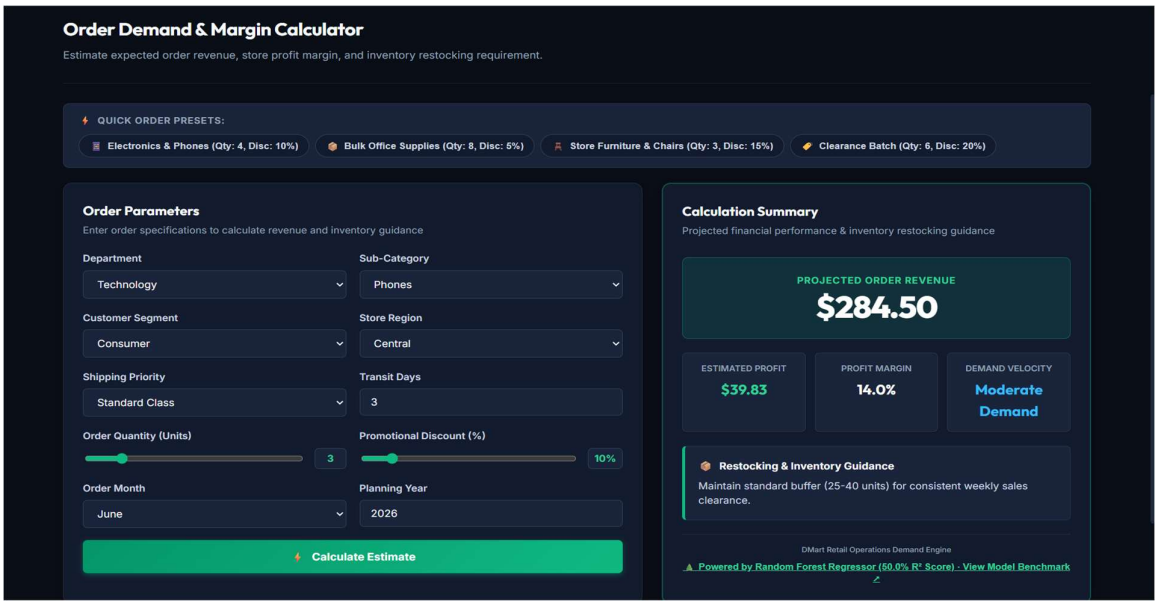


Figure 8.4.1: Interface of predicting model.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Smart Retail Sales Analysis and Demand Forecasting project demonstrates how a raw retail transaction dataset can be transformed into useful business information through a structured data analytics workflow. The implementation covers dataset loading, exploration, cleaning, feature engineering, exploratory data analysis, visualization, preprocessing, machine-learning model training, evaluation, model serialization, and new-input prediction.

The DMart sales dataset covering 2011–2017 is analyzed to understand Sales, Profit, product, category, customer-segment, regional, quantity, and discount-related patterns. Multiple regression models are trained with Sales as the target variable, and MAE, RMSE, and R^2 are used to evaluate their predictive performance. The Random Forest model is saved for reuse in the prediction stage.

The project also connects analytical findings with practical business use through dashboarding and a web-based prediction interface as described in the project requirements. Overall, the project demonstrates the practical application of Data Analysis Essentials concepts to a retail decision-making problem.

9.2 Limitations

- The current predictive implementation estimates Sales rather than implementing a dedicated multi-step time-series forecasting model.
- The supplied notebook does not contain the final printed numerical model evaluation outputs.
- The notebook uses a random train-test split rather than a chronological time-based split.
- The accuracy and usefulness of the analysis depend on the quality and completeness of the source dataset.
- Final Power BI dashboard and web-application screenshots are not contained in the supplied notebook.
- External factors affecting future retail demand are not directly represented by the available transaction features.

9.3 Future Enhancements

- Use dedicated time-series forecasting models with chronological validation for future demand estimation.
- Experiment with advanced ensemble and boosting methods where appropriate.
- Automate model retraining when new retail data becomes available.
- Integrate inventory optimization and reorder-level recommendations.
- Add interactive filters and drill-down capabilities to the dashboard.
- Deploy the prediction model through a production-ready web application.
- Monitor prediction performance over time and update the model when data patterns change.
- Include external demand drivers such as promotions, holidays, seasonality, and inventory information when available.

REFERENCES

1. DMart Sales Dataset(2011 – 2017), provided Excel Dataset: DMart_All_Data_2011_2017.xlsx
2. Project implementation notebook: DMart_Sales_Analysis_Project (1).ipynb.
3. Pandas Documentation – Python Data Analysis Library.
4. NumPy Documentation – Numerical Computing with Python.
5. Matplotlib Documentation – Python Data Visualization.
6. Scikit-learn Documentation – Machine Learning in Python.
7. Python Documentation – Python Programming Language.
8. Flask Documentation – Python Web Framework.

APPENDIX – SOURCE CODE

The following appendix contains the Python implementation extracted from the supplied project notebook. The code is presented to document the major stages of dataset loading, preprocessing, analysis, model training, evaluation, model saving, and prediction. Where notebook cells contain exploratory display statements or repeated analysis operations, they are retained as part of the project source record.

Code Cell 1

```
import pandas as pd
df = pd.read_excel("DMart_All_Data_2011_2017.xlsx", sheet_name="All Data")
print(df)
```

Code Cell 2

```
df.head()
```

Code Cell 3

```
df.tail()
```

Code Cell 4

```
df.shape
```

Code Cell 5

```
df.columns
```

Code Cell 6

```
df.dtypes
```

Code Cell 7

```
df.info()
```

Code Cell 8

```
df.describe()
```

Code Cell 9

```
df.describe(include="object")
```

Code Cell 10

```
df.isnull().sum()
```

Code Cell 11

```
df.columns
```

Code Cell 12
```
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
df["Order Date"].dtype
```

Code Cell 13
```
df["Order Date"].isnull().sum()
```

Code Cell 14
```
df["Order Date"].unique()[:10]
```

Code Cell 15
```
df["Ship Date"].unique()[:10] if "Ship Date" in df.columns else "Ship Date column not
available in all records"
```

Code Cell 16
```
if "Ship Date" in df.columns:
    df["Ship Date"] = pd.to_datetime(df["Ship Date"], errors="coerce")
df["Ship Date"].dtype if "Ship Date" in df.columns else "Not available"
```

Code Cell 17
```
df["Ship Date"].isnull().sum() if "Ship Date" in df.columns else "Not available"
```

Code Cell 18
```
df["Customer Name"].isnull().sum()
```

Code Cell 19
```
df["Customer Name"].nunique()
```

Code Cell 20
```
df["Segment"].unique()
```

Code Cell 21
```
mode = df["Segment"].mode()
df["Segment"] = df["Segment"].fillna(mode[0])
```

Code Cell 22
```
df["Segment"].unique()
```

Code Cell 23
```
df["Region"].unique()
```

Code Cell 24
```
mode = df["Region"].mode()
df["Region"] = df["Region"].fillna(mode[0])
```

Code Cell 25
```
df["Region"].unique()
```

Code Cell 26
```
df["Category"].unique()
```

Code Cell 27
```
df["Sub-Category"].unique()
```

Code Cell 28
```
df["Sales"].unique()[:20]
```

Code Cell 29
```
df["Sales"].dtype
```

Code Cell 30
```
df["Sales"].isnull().sum()
```

Code Cell 31
```
import matplotlib.pyplot as plt
plt.boxplot(df["Sales"].dropna())
plt.ylabel("Sales")
plt.show()
```

Code Cell 32
```
median = df["Sales"].median()
df["Sales"] = df["Sales"].fillna(median)
```

Code Cell 33
```
df["Sales"].isnull().sum()
```

Code Cell 34
```
df["Profit"].unique()[:20]
```

Code Cell 35
```
df["Profit"].dtype
```

Code Cell 36
```
df["Profit"].isnull().sum()
```

Code Cell 37
```
median = df["Profit"].median()
df["Profit"] = df["Profit"].fillna(median)
```

Code Cell 38
```
df["Profit"].isnull().sum()
```

Code Cell 39
```
df["Quantity"].unique()[:20]
```

Code Cell 40
```
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Quantity"].isnull().sum()
```

Code Cell 41

```
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median()).astype(int)
df["Quantity"].unique()[:20]
```

Code Cell 42

```
df["Discount"].unique()[:20]
```

Code Cell 43

```
df["Discount"] = pd.to_numeric(df["Discount"], errors="coerce")
df["Discount"].isnull().sum()
```

Code Cell 44

```
df["Discount"] = df["Discount"].fillna(df["Discount"].median())
df["Discount"].unique()[:20]
```

Code Cell 45

```
if "Feedback?" in df.columns:
    print(df["Feedback?"].unique())
    print(df["Feedback?"].isnull().sum())
else:
    print("Feedback column not available")
```

Code Cell 46

```
df.columns
```

Code Cell 47

```
drop_columns = [
    "Row ID",
    "Order ID",
    "Customer ID",
    "Product ID"
]
df = df.drop(columns=[c for c in drop_columns if c in df.columns], axis=1)
```

Code Cell 48

```
df.columns
```

Code Cell 49

```
df["Order_Year"] = df["Order Date"].dt.year
df["Order_Month"] = df["Order Date"].dt.month
df["Order_Day"] = df["Order Date"].dt.day
df["Order_Quarter"] = df["Order Date"].dt.quarter
```

Code Cell 50
```
df.columns
```

Code Cell 51
```
if "Ship Date" in df.columns:
    df["Shipping_Days"] = (df["Ship Date"] - df["Order Date"]).dt.days
else:
    df["Shipping_Days"] = 0
```

Code Cell 52
```
df.columns
```

Code Cell 53
```
df["Discount_Amount"] = df["Sales"] * df["Discount"]
df["Discount_Amount"].head()
```

Code Cell 54
```
df["Profit_Margin"] = (df["Profit"] / df["Sales"].replace(0, pd.NA)) * 100
df["Profit_Margin"] = df["Profit_Margin"].fillna(0)
df["Profit_Margin"].head()
```

Code Cell 55
```
df["Has_Discount"] = df["Discount"] > 0
df.columns
```

Code Cell 56
```
def sales_category(sales):
    if sales <= 100:
        return "Low"
    elif sales <= 500:
        return "Medium"
    else:
        return "High"
df["Sales_Category"] = df["Sales"].apply(sales_category)
```

Code Cell 57
```
df.columns
```

Code Cell 58
```
df["Sales"].sum().round(2)
```

Code Cell 59
```
df["Sales"].mean().round(2)
```

Code Cell 60

```
print("Minimum sales:", df["Sales"].min())
print("Maximum sales:", df["Sales"].max())
```

Code Cell 61

```
df["Profit"].sum().round(2)
```

Code Cell 62

```
df["Quantity"].mode()[0]
```

Code Cell 63

```
df["Sales"].plot(
   kind="hist",
   bins=20,
   edgecolor="black"
)
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.title("Distribution of Sales")
plt.show()
```

Code Cell 64

```
df.groupby("Order_Year")["Sales"].sum().round(2)
```

Code Cell 65

```
x = df.groupby("Order_Year")["Sales"].sum()
x.plot(kind="bar")
plt.xlabel("Order Year")
plt.ylabel("Total Sales")
plt.title("Total Sales by Year")
plt.show()
```

Code Cell 66

```
df.groupby("Order_Year")["Profit"].sum().round(2)
```

Code Cell 67

```
x = df.groupby("Order_Year")["Profit"].sum()
x.plot(kind="bar")
plt.xlabel("Order Year")
plt.ylabel("Total Profit")
plt.title("Total Profit by Year")
plt.show()
```

Code Cell 68
```
df["Category"].value_counts()
```

Code Cell 69
```
x = df["Category"].value_counts()
x.plot(kind="pie", autopct="%1.1f%%")
plt.ylabel("")
plt.legend()
plt.show()
```

Code Cell 70
```
df["Sub-Category"].value_counts()
```

Code Cell 71
```
df.groupby("Product Name")["Sales"].sum().sort_values(ascending=False).head(10)
```

Code Cell 72
```
df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
```

Code Cell 73
```
df.groupby("Category")["Profit"].sum().sort_values(ascending=False)
```

Code Cell 74
```
x = df.groupby("Category")["Sales"].sum()
x.plot(kind="bar")

plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.title("Sales by Category")
plt.show()
```

Code Cell 75
```
df["Segment"].value_counts()
```

Code Cell 76
```
x = df["Segment"].value_counts()
x.plot(kind="pie", autopct="%1.1f%%")
plt.ylabel("")
plt.legend()
plt.show()
```

Code Cell 77
```
df["Region"].value_counts()
```

Code Cell 78
```
df.groupby("Region")["Sales"].sum().sort_values(ascending=False)
```

Code Cell 79
```
df.groupby("Region")["Profit"].sum().sort_values(ascending=False)
```

Code Cell 80
```
x = df.groupby("Region")["Sales"].sum()
x.plot(kind="bar")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.title("Sales by Region")
plt.show()
```

Code Cell 81
```
df.groupby("Segment")["Sales"].mean().round(2)
```

Code Cell 82
```
df.groupby("Segment")["Profit"].mean().round(2)
```

Code Cell 83
```
df.groupby("Discount")["Profit"].mean().round(2)
```

Code Cell 84
```
pd.crosstab(df["Region"], df["Category"], values=df["Sales"], aggfunc="sum").round(2)
```

Code Cell 85
```
pd.crosstab(df["Segment"], df["Category"], values=df["Sales"], aggfunc="sum").round(2)
```

Code Cell 86
```
avg_sales = df.groupby("Quantity")["Sales"].mean()
avg_sales.plot(
   kind="line",
   marker="o",
   figsize=(7, 5)
)
plt.xlabel("Quantity")
plt.ylabel("Average Sales")
plt.title("Average Sales by Quantity")
plt.show()
```

Code Cell 87
```
df.columns
```

Code Cell 88

```
from sklearn.preprocessing import LabelEncoder
categorical_columns = [
    "Ship Mode",
    "Segment",
    "Country",
    "City",
    "State",
    "Region",
    "Category",
    "Sub-Category",
    "Sales_Category"
]
categorical_columns = [c for c in categorical_columns if c in df.columns]
le = LabelEncoder()
for column in categorical_columns:
    df[column] = le.fit_transform(df[column].astype(str))
df.head()
```

Code Cell 89

```
df = df.drop(
    columns=["Order Date", "Ship Date", "Product Name", "Customer Name", "Source",
"Feedback?", "Profit", "Discount_Amount", "Profit_Margin"],
    errors="ignore"
)
```

Code Cell 90

```
df = df.replace([float("inf"), float("-inf")], pd.NA)
df = df.fillna(0)
```

Code Cell 91

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scale_columns = [
    "Quantity",
    "Discount",
    "Order_Month",
    "Order_Day",
    "Order_Quarter",
    "Shipping_Days"
]
scale_columns = [c for c in scale_columns if c in df.columns]
df[scale_columns] = scaler.fit_transform(df[scale_columns])
```

Code Cell 92
```
X = df.drop("Sales", axis=1)
```

Code Cell 93
```
y = df["Sales"]
```

Code Cell 94
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Code Cell 95
```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Code Cell 96
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

Code Cell 97
```
from sklearn.tree import DecisionTreeRegressor
model_dt = DecisionTreeRegressor(random_state=42)
model_dt.fit(X_train, y_train)
```

Code Cell 98
```
from sklearn.ensemble import RandomForestRegressor
model_rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
model_rf.fit(X_train, y_train)
```

Code Cell 99
```
y_pred = model.predict(X_test)
```

Code Cell 100

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print("Linear Regression MAE:", mae)
print("Linear Regression RMSE:", rmse)
print("Linear Regression R2 Score:", r2)
```

Code Cell 101

```
y_pred_dt = model_dt.predict(X_test)
mae_dt = mean_absolute_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
r2_dt = r2_score(y_test, y_pred_dt)
print("Decision Tree MAE:", mae_dt)
print("Decision Tree RMSE:", rmse_dt)
print("Decision Tree R2 Score:", r2_dt)
```

Code Cell 102

```
y_pred_rf = model_rf.predict(X_test)
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)
print("Random Forest MAE:", mae_rf)
print("Random Forest RMSE:", rmse_rf)
print("Random Forest R2 Score:", r2_rf)
```

Code Cell 103

```
import pickle #SAVING THE ML MODEL OR TO CONVERT OBJECTS
with open("dmart_sales_model.pkl", "wb") as file:
    pickle.dump(model_rf, file)
```

Code Cell 104

```
df.columns
```

Code Cell 105

```
# Example prediction record
test_input = pd.DataFrame([{
    "Quantity": 3,
    "Discount": 0.10,
    "Order_Year": 2017,
    "Order_Month": 6,
    "Order_Day": 15,
```

```
    "Order_Quarter": 2,
    "Shipping_Days": 3,
    "Has_Discount": True
}])

# Add every feature expected by the trained model
for col in X_train.columns:
    if col not in test_input.columns:
        test_input[col] = 0

# Keep ONLY the features used during training
test_input = test_input[X_train.columns]

# Scale the same columns that were scaled during training
scale_columns = [
    "Quantity",
    "Discount",
    "Order_Month",
    "Order_Day",
    "Order_Quarter",
    "Shipping_Days"
]

scale_columns = [col for col in scale_columns if col in test_input.columns]

test_input[scale_columns] = scaler.transform(
    test_input[scale_columns]
)

# Prediction
prediction = model_rf.predict(test_input)
print("Predicted Sales:", round(float(prediction[0]), 2))
```